LiquidAI / LFM2.5-Embedding-350M

like 128 | Follow Liquid AI 4.06k

Sentence Similarity · sentence-transformers · Safetensors · 11 languages · lfm2 · liquid · lfm2.5 · edge · feature-extraction · custom_code · arxiv:2511.23404 · License: lfm1.0

Deploy · Copy to bucket NEW · Use this model

Liquid

[Try LFM](#) • [Docs](#) • [LEAP](#) • [Discord](#)

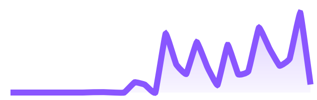
LFM2.5-Embedding-350M

We release two new **best-in-class multilingual retrieval** models:

- **LFM2.5-Embedding-350M** — A dense bi-encoder, one vector per document. Smallest, fastest index.
- [LFM2.5-ColBERT-350M](#) — A late-interaction model. One vector per *token*, matched via MaxSim. Higher accuracy and better generalization at the cost of index size.

Both models are 350M params and the first bidirectional members of the LFM family, built on [LFM2.5-350M-Base](#). They can be used as a **drop-in replacement** for your current RAG pipeline and

Downloads last month
23,353



Safetensors

Model size	0.4B params
Tensor type	BF16

Chat template

Files info

Inference Providers NEW

Sentence Similarity

This model isn't deployed by any Inference Provider.

Ask for provider support

Model tree for LiquidAI/LFM2.5-E...

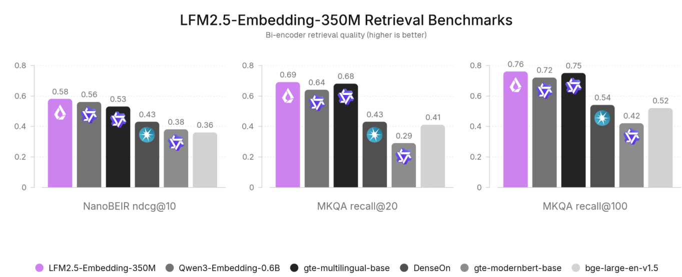
Base model	LiquidAI/LFM2.5-350M-Ba...
Finetuned (13)	this model
Finetunes	11 models
Quantizations	4 models

Spaces using LiquidAI/LFM2.5-... 4

LiquidAI/colbert-tool-selection

target fast, cheap, and reliable multilingual / cross-lingual search across 11 languages.

Find more details about the bidirectional architecture and training recipe in our blog post.



📄 Model details

Property	LFM2.5-Embedding-350M	LFM2.5-ColBERT-350M
Type	Dense bi-encoder (single vector)	Late interaction (per-token vectors)
Total parameters	~354M	~353M
Backbone	LFM2.5-350M-Base + bi-directional patches	LFM2.5-350M-Base + bi-directional patches
Layers	17 (10 conv + 6 attn + 1 pool)	17 (10 conv + 6 attn + 1 dense)
Vocabulary size	65,536	64,402
Output	1024-dim CLS vector	128-dim per token
Similarity	Cosine	MaxSim

⚖️ hakari-bench/leaderboard

🧠 Edu014/LFM2.5-Embedding

🧭 developerjeremylive/colbert-tool-s…

Collection including LiquidAI/LFM…

🎯 **Liquid Nanos** Collection
Library of tas… • 34 items • U. • △ 120

Paper for LiquidAI/LFM2.5-Embed…

LFM2 Technical Report
Paper • 2511.23404 • Publi… • △ 65

Property	LFM2.5-Embedding-350M	LFM2.5-ColBERT-350M
Training precision	BF16	BF16
License	LFM Open License v1.0	LFM Open License v1.0

Document length: 512 tokens

Supported languages: English, Spanish, German, French, Italian, Portuguese, Arabic, Swedish, Norwegian, Japanese, Korean.

Architecture:

```
SentenceTransformer(
  (0): Transformer({'max_seq_length': 512, '
  (1): Pooling({'word_embedding_dimension': 
)
```

Asymmetric prompts: `query:` for queries, `document:` for passages. They are stored in the model config and applied automatically via `prompt_name`.

We recommend LFM2.5-Embedding-350M and LFM2.5-ColBERT-350M for short-context retrieval use cases, such as:

- **E-commerce**: find products across many languages with semantic search at scale.
- **FAQ and support knowledge bases**: retrieve the right answer reliably across customer-facing surfaces.

- **On-device semantic search**: search files, emails, and notes locally on consumer hardware.
- **Enterprise knowledge assistants**: retrieve internal legal, financial, and technical documents across languages.

🏃 How to run

First, install `sentence-transformers`:

```
pip install -U sentence-transformers
```

Encoding queries and documents

Load LFM2.5-Embedding-350M and encode your queries and documents separately, using the matching prompt name on each side. Cosine similarity (or a normalized dot product) ranks documents against queries:

```
from sentence_transformers import SentenceTra

# Load the model (trust_remote_code applies
model = SentenceTransformer(
    "LiquidAI/LFM2.5-Embedding-350M",
    trust_remote_code=True,
)

queries = [
    "What is the capital of France?",
    "Which city is Japan's capital?",
]
documents = [
    "Paris is the capital and largest city o
    "Tokyo, officially the Tokyo Metropolis,
    "Berlin is the capital and largest city
```

```
]

# Encode with the matching prompt name; norm
q_emb = model.encode(queries,   prompt_name=
d_emb = model.encode(documents, prompt_name=

scores = q_emb @ d_emb.T  # shape: (n_querie
```

Always pass `prompt_name="query"` for queries and `prompt_name="document"` for passages — the model was trained with these prefixes, and omitting them silently degrades retrieval quality.

Flash Attention 2 (optional)

LFM2.5-Embedding-350M can run with FlashAttention-2 (requires `flash-attn` installed):

```
import torch
from sentence_transformers import SentenceTra

model = SentenceTransformer(
    "LiquidAI/LFM2.5-Embedding-350M",
    trust_remote_code=True,
    model_kwargs={"attn_implementation": "fla
)
```

Verified equivalent to the default within bf16 noise (multilingual NanoBEIR ndcg@10 within 0.002 across 11 languages). At the model's 512-token max length the speed gain is small (~5%); FA2 mainly helps memory and throughput if you fine-tune or run the backbone at longer contexts.

Fine-tuning

Standard `sentence-transformers` training works directly. Example with `MultipleNegativesRankingLoss`:

```
from datasets import Dataset
from sentence_transformers import (
    SentenceTransformer,
    SentenceTransformerTrainer,
    SentenceTransformerTrainingArguments,
)
from sentence_transformers.losses import Mul

model = SentenceTransformer("LiquidAI/LFM2.5
loss = MultipleNegativesRankingLoss(model)

train_ds = Dataset.from_dict({
    "query":    [...],
    "positive": [...],
    # optional: "negative": [...],
})

args = SentenceTransformerTrainingArguments(
    output_dir="out",
    num_train_epochs=1,
    per_device_train_batch_size=64,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    bf16=True,
    prompts={"query": "query: ", "positive":
)

trainer = SentenceTransformerTrainer(model=m
trainer.train()
```

Notes:

- Always pass the asymmetric prompts during training (the model was trained with them).

- For larger effective batches without OOM, swap `MultipleNegativesRankingLoss` for `CachedMultipleNegativesRankingLoss`.
- Save with `model.save_pretrained(...)`; the modeling file and `auto_map` are preserved so the patched behavior survives reloads.

📈 Performance

We highlight (= bold) the best bi-encoder and best late retriever for each language.

NanoBEIR Multilingual Extended — NDCG@10

`LiquidAI/nanobeir-multilingual-extended`. Multilingual retrieval capabilities.

Model	Type	AVG	ar	de	e
LiquidAI/LFM2.5-ColBERT-350M	late	**0.605**	**0.551**	**0.606**	0.6
LiquidAI/LFM2.5-Embedding-350M	dense	**0.577**	**0.529**	**0.581**	0.6
Qwen/Qwen3-Embedding-0.6B	dense	0.556	0.514	0.560	0.6
LiquidAI/LFM2-ColBERT-350M	late	0.540	0.491	0.563	0.6
Alibaba-NLP/gte-multilingual-base	dense	0.528	0.477	0.523	0.6
lightonai/GTE-ModernColBERT-v1	late	0.489	0.309	0.499	0.6
lightonai/LateOn	late	0.484	0.307	0.505	**0.6**

Model	Type	AVG	ar	de	e
lightonai/DenseOn	dense	0.432	0.178	0.474	**0.6**
Alibaba-NLP/gte-modernbert-base	dense	0.383	0.112	0.449	0.6
BAAI/bge-large-en-v1.5	dense	0.359	0.059	0.419	0.6

MKQA-11 — Recall@20

MKQA. Cross-lingual capabilities (subset of the 11 languages we target).

Model	Type	AVG	ar	de	e
LiquidAI/LFM2.5-ColBERT-350M	late	**0.694**	**0.608**	**0.709**	0.7
LiquidAI/LFM2.5-Embedding-350M	dense	**0.691**	**0.610**	**0.709**	0.7
Alibaba-NLP/gte-multilingual-base	dense	0.675	0.567	0.692	0.7
LiquidAI/LFM2-ColBERT-350M	late	0.646	0.554	0.696	0.7
Qwen/Qwen3-Embedding-0.6B	dense	0.638	0.520	0.671	0.7
lightonai/GTE-ModernColBERT-v1	late	0.459	0.092	0.532	0.7
lightonai/LateOn	late	0.454	0.157	0.492	**0.7**
lightonai/DenseOn	dense	0.435	0.165	0.482	**0.7**

Model	Type	AVG	ar	de	e
BAAI/bge-large-en-v1.5	dense	0.413	0.133	0.471	0.7
Alibaba-NLP/gte-modernbert-base	dense	0.295	0.060	0.333	0.7

Inference speed - llama.cpp

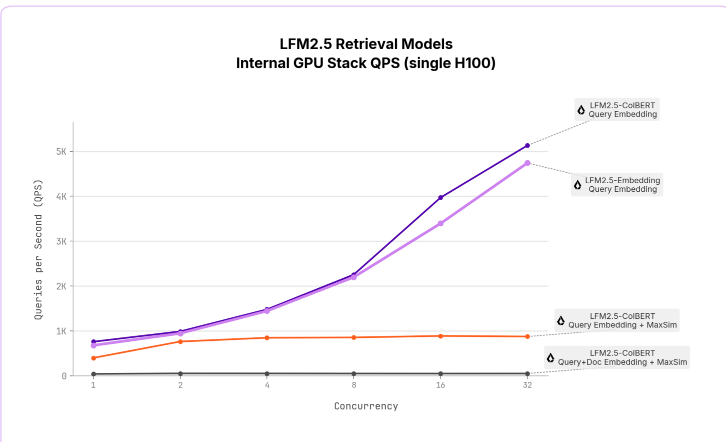
End-to-end latency on **MacBook Pro M4 Max** via **llama.cpp** at **fp16**, measured at **32-token queries** and **256-token documents**. `Docs cached` means that the document embeddings are pre-computed and looked up (from an index).

Model	Stage	Docs cached	p50	p95
LFM2.5-Embedding-350M	Query embedding	yes	7.3 ms	9.6 ms
LFM2.5-ColBERT-350M	Query embedding	yes	8.1 ms	8.5 ms
LFM2.5-ColBERT-350M	Query embedding + MaxSim	yes	8.2 ms	15.2 ms
LFM2.5-ColBERT-350M	Query embedding + Doc embedding + MaxSim	no	34.3 ms	36.3 ms

Both models LiquidAI/LFM2.5-ColBERT-350M-GGUF and LiquidAI/LFM2.5-Embedding-350M-GGUF are available on Hugging Face under different quantization schemas for llama.cpp.

Inference speed - Enterprise GPU

For large-scale production-grade enterprise deployments, we also experiment with an internal GPU stack to deliver extremely low-latency serving under high inbound load. We observe latencies as low as 1 ms.



Workload	Setup	p50	p95	p99
LFM2.5-Embedding-350M	Query embedding	1.5 ms	1.6 ms	1.7 ms
LFM2.5-ColBERT-350M	Query embedding	1.3 ms	1.4 ms	1.5 ms
LFM2.5-ColBERT-350M	Query embedding + MaxSim	2.5 ms	2.7 ms	2.8 ms
LFM2.5-ColBERT-350M	Query embedding + Doc embedding + MaxSim	22.8 ms	24.1 ms	26.4 ms

📬 Contact

- Got questions or want to connect? Join our Discord community.
- If you are interested in custom solutions with edge deployment, please contact our sales team.

Citation

```
@article{liquidai2025lfm2,
  title={LFM2 Technical Report},
  author={Liquid AI},
  journal={arXiv preprint arXiv:2511.23404},
  year={2025}
}
```